

LLMsVerifier

LLMsVerifier

Праверка. Маніторынг. Аптымізацыя.

Кароткае змесціва

LLMsVerifier — гэта платформа класа enterprise для праверкі, маніторынгу і аптымізацыі вялікіх моўных мадэляў ад розных пастаўшчыкоў, заснаваная на абавязковым тэсце праверкі «Ці бачыш ты мой код?», каб толькі тыя мадэлі, якія сапраўды працуюць, атрымлівалі статус прыдатных да выкарыстання ці экспарту.

Кароткае апісанне

Платформа Go, якая правярае, тэстуе, маніторыць і аптымізуе вялікія моўныя мадэлі ад некалькіх пастаўшчыкоў. Кожная мадэль павінна прайсці абавязковы тэст на бачнасць кода перад выкарыстаннем; затым яна праходзіць праверку на затрымку, стрымінг, выклік функцый, апрацоўку візуальных дадзеных і эмбедынгаў, а пасля гэтага экспартуе толькі правераныя канфігурацыі для інструментаў AI і CLI.

Падрабязнае апісанне

LLMsVerifier — гэта ўсебаковая платформа для праверкі, маніторынгу і аптымізацыі прадукцыйнасці LLM ад розных пастаўшчыкоў. Яе асноўны прынцып — *абавязковая праверка*, і кампанія не ідзе на кампрамісы: перад тым, як мадэль атрымае статус прыдатнай да выкарыстання ці будзе дазволена ў экспартаванай канфігурацыі, яна павінна пацвердзіць праходжанне тэсту «Ці бачыш ты мой код?», які здзяйсняе рэальныя HTTP-запыты да пастаўшчыка і аналізуе адказ на наяўнасць сапраўднага разумення, а не проста пераканаўчай імітацыі. Мадэль, якая не можа даказальна ўбачыць і зразумець ваш увод, проста ніколі не атрымае

сцяжок «прыдатная». Пасля гэтага бар'ера рухавік праверкі запуская поўны камплект тэстаў на здольнасці — існаванне, адказнасць, затрымку, стрымінг, выклік функцый, апрацоўку візуальных дадзеных, embeddings — а рухавік справаздач пераўтварае вынікі ў markdown і справаздачы JSON, з якімі можна працаваць.

Сістэма мае мадульную і падзеюарыентаваную архітэктuru, якая прапануе інтэрфейсы CLI, TUI, вэб і REST API на базе ядра з рухавіка праверкі, рухавіка справаздач і мэнэджара канфігурацый. Аднак платформа не абмяжоўваецца толькі праверкай. У яе дададзеныя дадатковыя ўзроўні: шаблон «Супервізар/Работнік» для дэкампіцыі задач на базе LLM, кіраванне кантэкстам праз слізгальнае акно з падсумоўваннем праз LLM, каб вельмі доўгія сесіі не гублялі кантэкст, рэзервавае захоўванне на воблаку, сістэма пераключэння на рэзервовыя каналы з засцерагальнікамі і маршрутызацыяй на аснове затрымкі. Навакольная інфраструктура мае прадукцыйную форму: шына падзей pub/sub, планаванне праз cron, выяўленне коштаў і абмежаванняў, база дадзеных vector для RAG і сістэма экспарту. Асаблівая сістэма брандынгу дадае суфікс (llmsvd) да кожнай згенераванай пары пастаўшчык/мадэль, каб правераны вывад можна было адразу адрозніць і ніколі не блытаць з неправераным — і толькі правераныя мадэлі трапляюць у экспартаваныя канфігурацыі для інструментаў AI CLI, такіх як OpenCode, Crush і Claude Code. Платформа пастаўляецца з тымі аперацыйнымі інструментамі, якія сапраўды патрэбныя камандам у прадукцыйным асяроддзі: разгортванне Docker/Kubernetes/Helm, маніторынг Prometheus/Grafana, аўтэнтыфікацыя праз LDAP/SSO і шыфраванне даных SQLCipher.

Чаму мы яе стварылі

Таму што праверка толькі праз канфігурацыю — ненадзейная: ключ API можа скончыцца, мадэль можа быць знята з эксплуатацыі, а файл канфігурацыі нічога не гаворыць пра рэальную затрымку, рэальныя памылкі ці пра тое, ці можа мадэль сапраўды ўбачыць і зразумець ваш увод. LLMsVerifier замяняе прынцып «гэта ёсць у канфігурацыі, значыць павінна працаваць» на доказы: толькі тыя мадэлі, якія даказальна адказваюць правільна, атрымліваюць статус прыдатных і экспартуюцца.

Змесціва

Чаму гэта змяняе гульнію

Гэта робіць флаты LLM *надзейнымі* — слова, якое рэдка заслугоўваецца ў прасторы, дзе канфігурацыі хлусяць праз недаказанне. Замест таго каб спадзявацца, што настроеная мадэль будзе працаваць, каманды атрымліваюць абавязковую, правераную гарантыю, што кожная мадэль у працы прайшла рэальную верыфікацыю, а маніторынг, пераключэнне на рэзервовыя сродкі і экспарт толькі правераных мадэляў замыкаюць цыкл ад доказу да прадукцыйнага асяроддзя. У экасістэме Helix гэта становіцца адзінай крыніцай праўды для метадавых мадэляў LLM, пастаўшчыкоў і верыфікацыі: іншыя сэрвісы (у тым ліку HelixTranslate) звяртаюцца да яе, і ўся платформа атрымлівае адзін сумленны адказ на пытанне «якія мадэлі сапраўды працуюць зараз?» замест таго, каб кожная каманда падтрымлівала ўласныя надзеі.

Што новага

- **Абавязковая верыфікацыя «Ці бачыш ты мой код?»** — рэальны, падтрыманы HTTP бар’ер разумення, які павінна прайсці мадэль, перш чым яе можна будзе выкарыстоўваць; галоўная адметнасць прадукту і прычына, па якой нічога неправеранага не праскочыць.
- **Экспарт канфігурацый толькі з праверанымі мадэлямі** — згенераваныя канфігурацыі для інструментаў AI CLI змяшчаюць *толькі* мадэлі, якія прайшлі верыфікацыю, таму канфігурацыя, якую вы адпраўляеце, не зможжа ціха вярнуць пашкоджаную мадэль.
- **Сістэма суфіксаў брэйдінгу (llmsvd)** — кожны згенераваны пастаўшчык/мадэль мае адсочвальны суфікс, што робіць правераную праведнасць бачнай усюды, куды трапляе вывад.
- **Дэтэкцыя магчымасцей** для шматлікіх агентаў і пастаўшчыкоў CLI — сістэма вызначае тыпы стрымінгу (SSE, WebSocket, JSONL, EventStream), кампрэсію і паводзіны кэшавання, замест таго каб рабіць здагадкі.
- **Устойлівае пераклучэнне на рэзервовыя сродкі** — аўтаматычныя выключальнікі, маршрутызацыя на аснове затрымкі, якая перанакіроўвае трафік, калі час да першага токена перавышае парог, праверкі здароўя і размеркаванне нагрузкі з улікам вагі забяспечваюць рэактыўнасць флоту, калі асобныя пастаўшчыкі пачынаюць «хісціцца».
- **Даўгатэрміновая аўтаномнасць** — шаблон раскладання на Супервізара і Работнікаў разам з кропкамі захавання стану і інтэграцыяй памяці падтрымліваюць працяглыя сесіі, якія інакш перавысілі б кантэкст.
- **Інтэграцыя з RAG / vector-DB** для паляпшэння кантэксту на аснове рэальных даных.

Галоўныя тэхнічныя выклікі і як мы іх пераадолелі

- **Доказ таго, што мадэль сапраўды працуе, а не проста наладжана.** Уся сутнасць і найбольшая складанасць. Вырашана праз абавязковы тэст бачнасці кода, які робіць рэальныя запыты да API і аналізуе адказы на разуменне, падтрыманы шырокім наборам тэстаў на магчымасці, — а потым адмаўляецца экспартаваць усё, што не прайшло праверку, так што ў прадукцыйнае асяроддзе трапляе толькі тое, што даказана, а не проста наладжана.
- **Надзейнасць пры працы з ненадзейнымі трэцімі пастаўшчыкамі.** Вырашана з дапамогай аркестратара пераключэння на рэзервовыя сродкі, які лічыць нястабільнасць пастаўшчыкоў нормай: аўтаматычныя выключальнікі пазначаюць пастаўшчыка як дэградаваны пасля N збоў за M секунд, маршрутызацыя на аснове затрымкі адводзіць трафік ад павольных канчатковых кропак, перыядычныя праверкі здароўя кантралююць аднаўленне, а размеркаванне нагрузкі з улікам вагі балансуе паміж эканамічнымі і прэміяльнымі мадэлямі.
- **Падтрымка вельмі працяглых аўтаномных сесій.** Вырашана праз шаблон раскладання на Супервізара і Работнікаў, які падзяляе вялікую працу на кіравальныя часткі, перыядычнае захаванне стану ў воблачнае сховішча, каб прагрэс не губляўся пры перапынку, і шматслойнае кіраванне кантэкстам (акно слізгацення + зводка LLM + RAG), каб мадэль захоўвала сувязь, не танучы ў токенах.
- **Размнажэнне пастаўшчыкаў.** Вырашана схаваннем шматлікіх адаптараў Go для кожнага пастаўшчыка за адзіным агульным інтэрфейсам, прычым рэальныя канчатковыя кропкі пералічваюцца цэнтралізавана, — так даданне новага пастаўшчыка становіцца лакальнай зменай, а не хваляй па ўсім кодзе.

Змесціва

Тэхналагічны стэк

- **Go** — абраны ў якасці асноўнай мовы платформы за сваю канкурэнтнасць; кіруе шматпатокавым рухавіком праверкі, які можа аналізаваць шмат мадэляў паралельна, а таксама навакольнымі сэрвісамі.
- **Gin** — абраны ў якасці REST API-сервера, які забяспечвае JWT-аўтэнтыфікацыю, абмежаванне хуткасці і канчатковыя кропкі WebSocket/SSE.
- **SQLite + SQLCIPHER** — абраны для ўбудаванага захоўвання з шыфраваннем на ўзроўні базы даных,

паколькі дадзеныя праверкі (ключы, вынікі) з'яўляюцца канфідэнцыйнымі і павінны быць зашыфраваны ў спакоі па змаўчанні.

- **Redis** — абраны ў якасці пласта кэшавання для хуткага доступу да актыўных праверак і метададзеных.
- **RabbitMQ + Kafka** — абраны для падтрымкі падзеі-арыентаванай архітэктур: абмен паведамленнямі і стрымінг, якія раздзяляюць вытворцаў і спажыўцоў па ўсёй платформе.
- **gRPC + Protocol Buffers** — абраны для строга тыпізаванай міжсэрвіснай камунікацыі і транспарту падзей паміж кампанентамі.
- **QUIC / HTTP-3 (quic-go)** — абраны для падтрымкі сучаснага транспарту (у дакументацыі сховішча пазначана абмежаваная даступнасць HTTP/3-правайдара — гэта магчымасць, а не ўніверсальнае сцвярджэнне).
- **JWT + LDAP/NTLM** — абраны для карпаратыўнай аўтэнтыфікацыі, каб платформа інтэгралася з існуючай карпаратыўнай ідэнтычнасцю (SSO/SAML/OIDC абвешчаны ў дакументацыі).
- **Viper (канфігурацыя), Logrus (лагіраванне), Brotli/compress (сцісканне)** — аперацыйная інфраструктура: гнуткая канфігурацыя, структураванае лагіраванне і сцісканне нагрузкі.
- **Angular** — абраны для вэб-аднастаронкавай праграмы, візуальнага фасаду праверкі і маніторынгу.
- **Python + JavaScript SDK** — абраны для першакласнага доступу каманд кліентаў, дакументаваны праз OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — абраны для разгортвання ў прадукцыйным асяроддзі з маніторынгам здароўя і аўтамаштабаваннем, каб парк праверак маштабаваўся як любы сучасны сэрвіс.
- **Prometheus + Grafana** — абраны для збору метрык і стварэння панэляў кіравання, каб здароўе самой платформы было настолькі ж назіральным, як і мадэлі, якія яна кантралюе.
- **Testify (Go) + node -test/jsdom (вэб)** — абраны для шматузроўневага тэсціравання Go-ядра і вэб-фронтэнда.

Стан і заўвагі аб шчырасці

- **Стан: бэта.** Крынічны код Go рэалізуе рэальную HTTP-правярку (адзін састарэлы дакумент, які апісвае правярку як канфігурацыйную, з'яўляецца пажаданнем і састарэлым — аўтарытэтным з'яўляецца код).
- **Ліцэнзія: пакуль не вызначана.** У README пазначана MIT, а ў Dockerfile — Apache-2.0 — вырашыць пытанне перад публікацыяй.

- Колькасць правайдараў: у README пазначана “12 адаптараў”, але ў каталогу правайдараў пералічана каля 26 — разглядаць як “12+ / больш у працэсе”. Існуе шмат файлаў са статусам “FINAL/COMPLETE”; аўтарытэтнымі з’яўляюцца код, дакументацыя і go.mod.
- Рэпазіторый знаходзіцца ў арганізацыі vasic-digital, але функцыянальна з’яўляецца даверным пластом кластара Helix LLM-інфраструктуры.

Прыярытэтны ўзровень: Helix-прыярытэтны (кластар LLM-інфраструктуры; адзіная крыніца праўды для метададзеных LLM/правайдара/правяркi). Пасля HelixTrack.